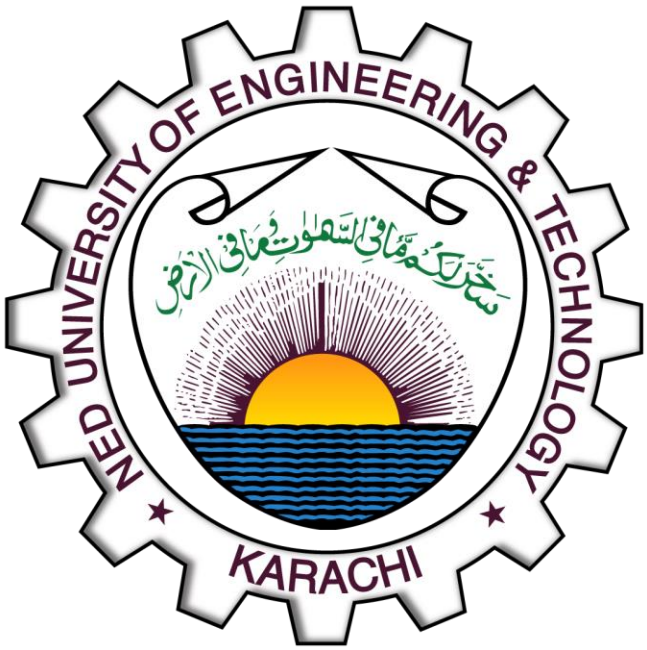
DATA STRUCTURE ALGORITHMS & APPLICATIONS

[CT-159]



NAME: TAHA AHMED MALLICK

ROLL NO: CT-25183

DEPARTMENT: BCIT **BATCH:** 2025

YEAR & SECTION: SSCS-D

LAB : #01

QUESTION#1:

CODE:

```cpp
#include <iostream>
using namespace std;

int main()
{
    int m, n;
    cout << "ROW: ";
    cin >> m;
    cout << "COL: ";
    cin >> n;
    int arr[m][n];
    cout << "Enter elements\n";
    for (int i = 0; i < m; i++)
        for (int j = 0; j < n; j++)
            cin >> arr[i][j];
    cout << "\n\nThe entered elements\n";
    for (int i = 0; i < m; i++)
    {
        for (int j = 0; j < n; j++)
            cout << arr[i][j] << " ";
        cout << endl;
    }
    cout << "\nCOL-majored 1D representation\n";
    int arr2[m + n];
    for (int i = 0; i < n; i++)
        for (int j = 0; j < m; j++)
            arr2[i*m +j] = arr[j][i];
    for (int i = 0; i < (m + n + 1); i++)
        cout << arr2[i] << " ";
    return 0;
}
```

OUTPUT:

```
ROW: 3
COL: 2
Enter elements
1 2
3 4
5 6


The entered elements
1 2
3 4
5 6

COL-majored 1D representation
1 3 5 2 4 6
```

QUESTION#2:

CODE:

```
#include <iostream>
using namespace std;

int main() {
    double marks[][5] = {
        {3.66, 3.33, 4.0, 3.0, 2.66},
        {3.33, 3.0, 3.66, 3.0, -1},
        {4.0, 3.66, 2.66, -1, -1},
        {2.66, 2.33, 4.0, -1, -1},
        {3.33, 3.66, 4.0, 3.0, 3.33}
    };
    string names[] = {"Ali", "Hiba", "Asma", "Zain", "Faisal"};

    for (int i = 0; i < 5; i++) {
        double avg = 0;int count = 0;
        for (int j = 0; j < 5; j++)
            if (marks[i][j] != -1) {
                avg += marks[i][j];
                count++;
            }
        avg /= count;
        cout << "GPA of " << names[i] << " in " << count << " courses is: " << avg <<
endl;
    }

    return 0;
}
```

OUTPUT:

```
GPA of Ali in 5 courses is: 3.33
GPA of Hiba in 4 courses is: 3.2475
GPA of Asma in 3 courses is: 3.44
GPA of Zain in 3 courses is: 2.99667
GPA of Faisal in 5 courses is: 3.464
```

QUESTION#3:

CODE:

```cpp
#include <bits/stdc++.h>
using namespace std;

class MedianFinder {
public:
    vector<int> list;
    MedianFinder() {

    }

    void addNum(int num) {
        list.push_back(num);
    }

    double findMedian() {
        sort(list.begin(), list.end());
        int n = list.size();
        double med = double(list[(n-1)/2]+list[n/2])/2;
        return med;
    }
};

/**
 * Your MedianFinder object will be instantiated and called as such:
 * MedianFinder* obj = new MedianFinder();
 * obj->addNum(num);
 * double param_2 = obj->findMedian();
 */

int main() {
    MedianFinder *medfinder = new MedianFinder();
    medfinder->addNum(2);
    medfinder->addNum(1);
    cout << medfinder->findMedian() << endl;
    medfinder->addNum(3);
    cout << medfinder->findMedian() << endl;
    return 0;
}
```

OUTPUT:

```
1.5
2
```

QUESTION#4:

CODE:

```cpp
#include <iostream>
using namespace std;

int binSearch(int arr[], int n, int t) {
    int l = 0, r = n-1;
    while(l <= r) {
        int m = l + (r-l)/2;
        if (arr[m] == t)
            return m;
        else if (t < arr[m])
            r = m - 1;
        else
            l = m + 1;
    }
    return -1;
}

int main() {
    int n, t;
    cout << "LEN: ";
    cin >> n;
    int arr[n];
    cout << "ENTER ELEMENTS: ";
    for (int i = 0; i < n; i++)
        cin >> arr[i];
    cout << "ENTER TARGET: ";
    cin >> t;
    int index = binSearch(arr, n, t);
    (index == -1) ? (cout << "TARGET N/A\n") : (cout << "TARGET AT: " << index <<
endl);
    return 0;
}
```

OUTPUT:

```
LEN: 6
ENTER ELEMENTS: -1 0 3 5 9 12
ENTER TARGET: 9
TARGET AT: 4
```

QUESTION#5:

CODE:

```
#include <iostream>
using namespace std;

bool binSearch(int **arr, int m, int n, int t) {
    int l = 0, r = m*n-1;
    while (l <= r) {
        int mid = l + (r-l)/2;
        int row = mid /n;
        int col = mid %n;
        if (t == arr[row][col])
            return true;
        else if (t < arr[row][col])
            r = mid - 1;
        else
            l = mid + 1;
    }
    return false;
}

int main()
{
    int m, n;
    cout << "ROW: ";
    cin >> m;
    cout << "COL: ";
    cin >> n;
    int** arr = new int*[m];
    cout << "Enter elements\n";
    for (int i = 0; i < m; i++) {
        arr[i] = new int[n];
        for (int j = 0; j < n; j++)
            cin >> arr[i][j];
    }
    cout << "\n\nThe entered elements\n";
    for (int i = 0; i < m; i++)
    {
        for (int j = 0; j < n; j++)
            cout << arr[i][j] << " ";
        cout << endl;
    }
    int t;
    cout << endl << "Target: ";
    cin >> t;
    binSearch(arr, m, n, t) ? (cout << "found the element.") : (cout << "couldn\'t
find the Specified element.");
    cout << endl;
    return 0;
}
```

OUTPUT:

```
ROW: 3
COL: 4
Enter elements
1 3 5 7
10 11 16 20
23 30 34 60


The entered elements
1 3 5 7
10 11 16 20
23 30 34 60

Target: 3
found the element.
```